



Jaxer Chat Walkthrough

This sample ships with code updated for Jaxer 1.0 and Aptana Studio 1.2 and is a good source of information but the tutorial has not yet been revised to match.

Contents

- [Introduction](#)
 - [Viewing the sample application](#)
 - [Creating a new account](#)
- [What just happened](#)
 - [Jaxer page lifecycle](#)
- [Initial page load](#)
 - [Processing details](#)
- [Callbacks made easy](#)
 - [On the client](#)
 - [On the server](#)
- [Inside the Jaxer chat application](#)
 - [Architectural distinction](#)
 - [Side-by-side comparison](#)
- [Jaxer runat and proxy reference](#)
 - [Basic runat values](#)
 - [Advanced runat values](#)
 - [Programmatic runat configuration](#)
 - [Recommended style](#)
 - [Alternate syntax](#)
- [jaxer:include reference](#)
 - [Why use server-side includes?](#)

Ajax, meet server.

Ever wish you could use your Ajax, DOM, and Javascript skills on the server? Now you can!

Download Jaxer for your own server now!

Ajax everywhere. **aptana Jaxer**

Introduction

Your Jaxer installation includes a demo of an online multi-room chat application. This demo application includes much of the major functionality that you would find in a typical online chat app:

- Creating a new user account
- Logging in to chat
- Creating a new chat room
- Entering a chat room
- Chatting with other users
- Exiting a chat room
- Logging out of chat

This section briefly explains how the demo chat application works from a user perspective. After learning how a user would use chat, you'll learn about the Jaxer architecture and how to use it to create an application similar to chat.

Viewing the sample application

Your Jaxer installation (whether as a standalone Jaxer Package or in Aptana Studio) includes this simple chat application.

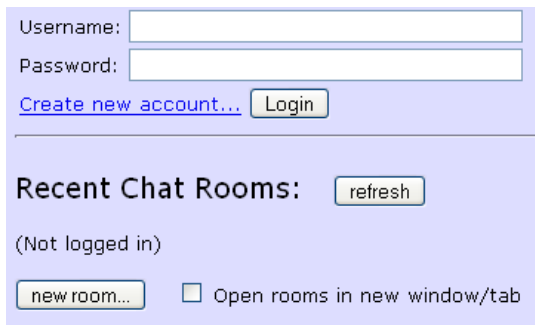
To access the chat application from within Aptana Studio:

1. In the Samples View, expand the **Aptana Jaxer** folder.
2. Select the **chat** example.
3. Click either the **Preview Sample** button  to do a quick preview of the sample, or click the **Import Sample** button  to import the sample as a project into your workspace.

To access the chat application from a standalone Jaxer Package installation:

1. Navigate to your Jaxer installation folder, and double-click the **StartServers.bat** file.
2. In your web browser, navigate to the following URL: <http://localhost:8081/aptana/>
3. From the column on the left, click the **Apps and Tools** link.
4. Click the **Chat** link.

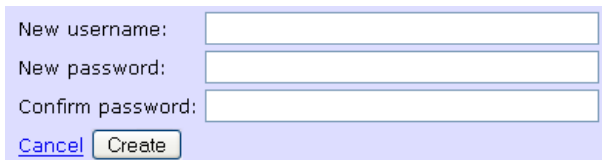
You can now experiment with creating new users, new chat rooms, and chatting in the demo application (shown below):



Creating a new account

As with many web applications, new chat users need to create new accounts before they can log into and use chat. To create a new user account:

1. On the main chat screen, click the **Create new account...** link to go to the new account screen.



2. On the new account screen, type your user information:
 1. In the **New username** field, type your username.
 2. In the **New password** field, type your password (at least 6 characters).
 3. In the **Confirm** password field, re-type your password.
3. Click the **Create** button to create your new account.

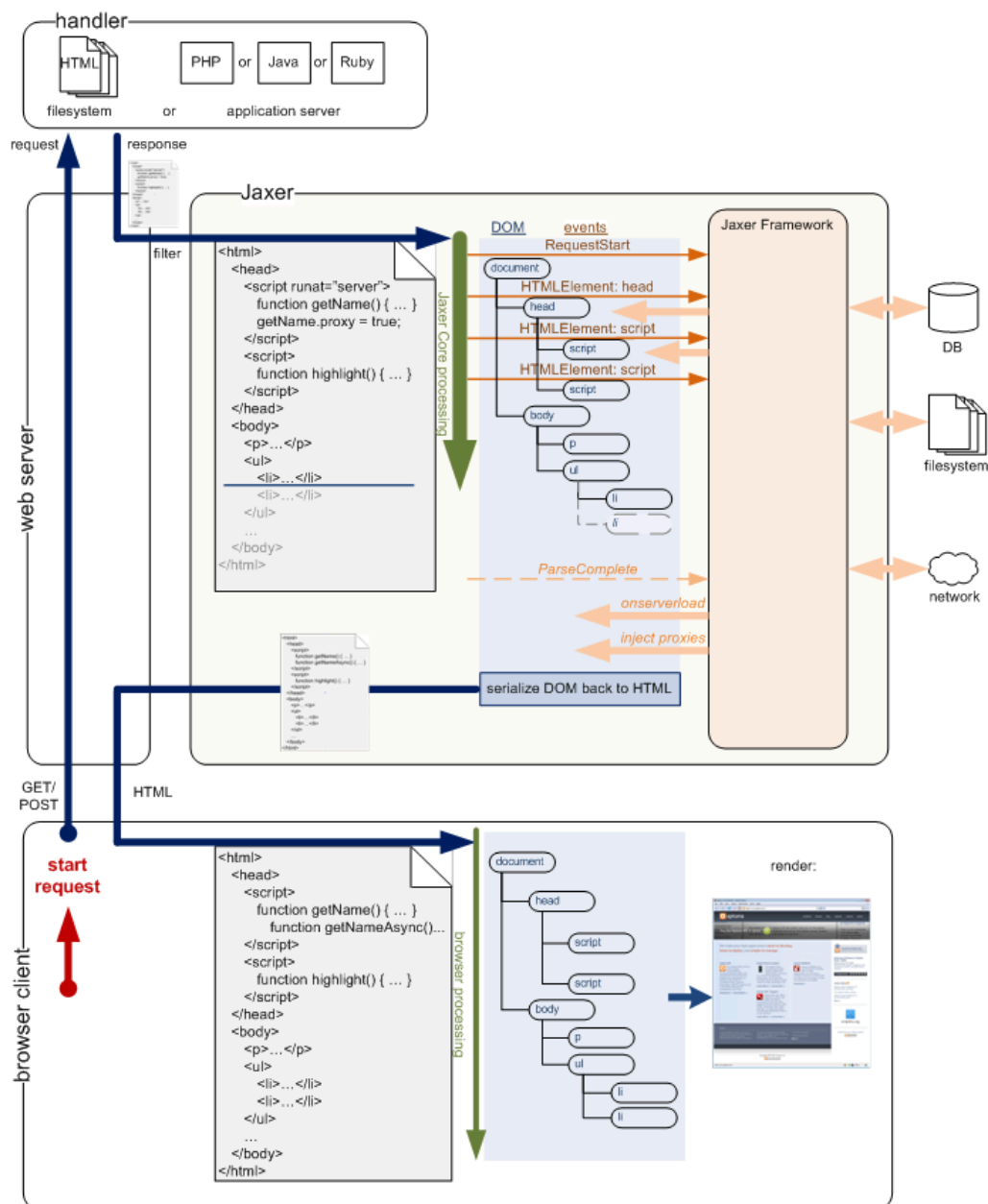
The Jaxer chat room creates your new account and automatically logs you in to chat.

What just happened?

When you request the chat application's page, and when you login, a series of events takes place. Some of these events happen during page preparation, some happen locally in your browser, and some involve communicating back to the Jaxer server and — through the Jaxer server framework — to the database and the file system.

Jaxer page lifecycle

The following diagram shows a simple overview of the Jaxer page processing chain:



Lifecycle events

The following list describes what happens when Jaxer processes a web page:

1. The client (browser) sends a request to Apache, which either handles the request itself or delegates it to a handler such as PHP, Ruby, Java, etc.
2. Apache either retrieves the static HTML document from disk, or receives a dynamic HTML document from the handler.
3. Jaxer acts as an output (post-process) filter that receives the document from Apache, and starts to parse and execute it, just as a browser would, but on the server side. The DOM is created, JavaScript code designated to run on the server is executed, and so on until the entire document is consumed.
4. The result is a DOM modified by the Jaxer Framework and by the developer: in particular, server-side client-callable functions are automatically replaced by proxies. Some important side-effects include storing designated JavaScript functions as callbacks and persisting session-type data.
5. The new DOM is serialized (without any server-side JavaScript) as a new HTML document and streamed out to the client as usual.
6. The client receives the HTML document and the processing continues, recreating the DOM from the provided HTML and this time executing the client-side JavaScript remaining on the page as normal.

The client-side JavaScript served to the browser may contain proxy function stubs, that correspond to (and have the same name as) certain server-side functions tagged by the developer to be callable from the client. By using the proxy architecture, Jaxer is able to provide powerful server-side capabilities accessible from the client in an entirely native and seamless manner.

The next section examines these events in more detail.

Initial page load

The initial page load processing for Jaxer is similar to other web architectures, in that the request to the browser is passed through both a web server (in this case Apache) and an Application server layer (in this case the Jaxer apache handler/filter) before being presented to the client browser.

Processing details

Preparing the database schema

When the Jaxer server receives the page to process, several behind the scenes actions take place, among them:

- Jaxer performs any server side activity, such as database access.
- The page is automatically instrumented with server callback capabilities by binding certain client functionality to server-side scripts.

The enabling mechanism for controlling the server callback functionality is the `runat` attribute, an additional attribute that Jaxer recognizes on in-line or external script tags.

Even without the presence of `runat` attributes on your script tags, Jaxer will still take certain actions on your code, such as persisting session-type data and inserting the Aptana namespace object into the DOM.

Examine the code of the first page of HTML sent, **index.html**, to see what it does:

The first script tag loads the file **setup.js**.

```
1. <script type="text/javascript" src="http://jaxer.org/tutorials/jaxer-include/setup.js" runat="server"></script>
```

The `runat` attribute provided to the script tag tells the Jaxer server that the contents of this script should execute server side and not be presented to the client browser. This means that if you view the source on the index page after it loads in your browser, you will not see any of the contents of the **setup.js** file (shown below).

```
01. (function() {
02.
03. // Jaxer 0.9.7 or earlier // if (Jaxer.application.get("isInitialized")) return;
04. if (Jaxer.application.isInitialized) return;
05.
06. var sql;
07.
08. sql = "CREATE TABLE IF NOT EXISTS users " +
09.       "( id INTEGER PRIMARY KEY AUTO_INCREMENT " +
10.       ", username VARCHAR(255) NOT NULL " +
11.       ", password VARCHAR(255) NOT NULL " +
12.       ", created DATETIME NOT NULL " +
13.       ", last_login DATETIME NOT NULL " +
14.       ", UNIQUE (username) " +
15.       ")";
16.
17. Jaxer.DB.execute(sql);
18.
19. sql = "CREATE TABLE IF NOT EXISTS messages " +
20.       "( id INTEGER PRIMARY KEY AUTO_INCREMENT " +
21.       ", room_id INTEGER NOT NULL " +
22.       ", sent DATETIME NOT NULL " +
23.       ", user_id INTEGER NOT NULL " +
24.       ", contents TEXT NOT NULL " +
25.       ")";
26.
27. Jaxer.DB.execute(sql);
28. // Jaxer 0.9.7 or earlier // Jaxer.application.set("isInitialized", true);
29. Jaxer.application.isInitialized = true;
30.
31.})();
```

The script file **setup.js** checks the application level container object and looks for a stored variable called `isInitialized`. If it already exists, the application has already been run and can return control to the original page **index.html**. However, if the variable `isInitialized` is not found, this is the first time the application has been run, and some SQL DDL is executed to bootstrap the database, which creates the required schema to run the application. After the SQL executes and the database is prepared, the `isInitialized` flag is set to true, and the processing returns to the index page.

You now have a schema ready for use by the application. Note that by using the Jaxer framework modules, you can accomplish all of this using only JavaScript.

Adding server- and client-side functionality

The next script tag that you encounter uses the `runat='both'` value to tell the server that the code is to be run on both the browser and the server. The server-side functions will also be cached in association with this page so that they can be used by any server-side callback functions.

```
01. <script type="text/javascript" runat="both">
02.
03. function $(id) { return document.getElementById(id); }
04. function fromTemplate(idTarget, idSource) { $(idTarget).innerHTML = $(idSource).innerHTML.replace(/(\W)\$id\=/g, "$1id="); }
05.
06. function showRooms(rooms)
07. {
08.     if (rooms == null)
09.     {
10.         $("rooms").innerHTML = "(Not logged in)";
11.     }
12.     else
13.     {
14.         var htmlStrings = [];
15.         for (var i = 0; i < rooms.length; i++) {
16.             var room = rooms[i];
17.             var html = $("roomTemplate").innerHTML.replace(/\\(\\w+)\\$/g, function(match, keyword){
18.                 return room[keyword];
19.             });
20.             htmlStrings.push(html);
21.         }
22.         $("rooms").innerHTML = "\\n" + htmlStrings.join("\\n") + "\\n";
23.     }
24. }
25.
26.
27. </script>
```

The functions in this script block will be needed on both the server and the client in order to populate the dynamic section of the DOM server-side for the initial page presentation, and then to update the DOM client-side as needed in response to user actions. Note that the **same** DOM manipulation code can be used on both client and server sides, so the DOM is consistent and the code is shorter and more maintainable.

The next script we encounter is only needed server-side:

```
1. <script type="text/javascript" src="http://jaxer.org/tutorials/jaxer-include/_authentication.js" runat="server"></script>
```

The script **_authentication.js** contains some functions for use both now during page preparation and later during callbacks. These functions will be cached automatically by using `runat='server'`. (If you know you won't need them during callbacks, and you'd like to optimize performance and memory a bit, you can use

runat='server-nocache' instead.)

```

01. function getAuthenticatedUser()
02. {
03.     // Jaxer 0.9.7 or earlier // var username = Jaxer.session.get("username");
04.     var username = Jaxer.session.username;
05.     if (typeof username == "undefined") return null;
06.     var rs = Jaxer.DB.execute("SELECT * FROM users WHERE username = ?", [username]);
07.     if (rs.rows.length == 0)
08.     {
09.         return null;
10.     }
11.     return rs.rows[0];
12. }
13.
14. function makeAuthenticated(user)
15. {
16.     // Jaxer 0.9.7 or earlier // Jaxer.session.set("username", user.username);
17.     Jaxer.session.username = user.username;
18. }
19.
20. function initAuthentication()
21. {
22.     try
23.     {
24.         var user = getAuthenticatedUser();
25.         if (user)
26.         {
27.             fromTemplate('loginComponent', 'loginAuthenticated');
28.             $('authenticatedUsername').innerHTML = user.username;
29.             Jaxer.clientData.isAuthenticated = true;
30.         }
31.         else
32.         {
33.             fromTemplate('loginComponent', 'loginRequest');
34.             Jaxer.clientData.isAuthenticated = false;
35.         }
36.     }
37.     catch (e)
38.     {
39.         Jaxer.Log.error("Error trying to get authenticated user: " + e);
40.     }
41. }

```

This code was encapsulated in an external .js file since it will be used on multiple pages where authentication is needed, guaranteeing that the same logic will be used consistently. Note that it only runs on the server, and in fact cannot be called from client-side JavaScript functions, for security reasons. Next we'll see an example of code that should be called from the client.

Specifying where your code runs

This really gets to the power of the Jaxer architecture: by being able to specify whether the code runs on the server or the client or both, and allowing the server-side functions to be seamlessly called from the client, we can build highly consistent code that crosses the client/server boundary with little effort, thus avoiding the requirement in other web stacks to use templating constructs, emit JavaScript from a different language, create handler pages for your server-side callbacks, etc. In Jaxer it is **all** DOM and JavaScript.

The next script is an in-line script with a runat attribute set to execute on the server.

```

01. <script type="text/javascript" runat="server">
02.
03.     function getRooms()
04.     {
05.         var user = getAuthenticatedUser();
06.         if (user == null) return null;
07.         var rs = Jaxer.DB.execute("SELECT MIN(sent) AS started, MAX(sent) AS last, room_id FROM messages GROUP BY room_id");
08.         // Fix up dates because SQLite does not (reliably?) return the declared column type on aggregates
09.         for (var i=0, len = rs.rows.length; i<len; i++)
10.         {
11.             var row = rs.rows[i];
12.             if (typeof row.started == "number") row.started = formatShortDate(new Date(row.started));
13.             if (typeof row.last == "number") row.last = formatShortDate(new Date(row.last));
14.         }
15.         return rs.rows;
16.     }
17.     getRooms.proxy = true;
18.
19.     function numericPad(num,width)
20.     {
21.         return num > Math.pow(10,width) ? num.toString() : (Math.pow(10,width) + num).toString().substring(1);
22.     }
23.
24.     function formatShortDate(dt)
25.     {
26.         var dd = numericPad(dt.getDate(),2);
27.         var mm = numericPad(dt.getMonth()+1,2);
28.         var yr = numericPad(dt.getFullYear(),2);
29.
30.         var hr = numericPad(dt.getHours(),2);
31.         var mn = numericPad(dt.getMinutes(),2);
32.         var ss = numericPad(dt.getSeconds(),2);
33.
34.         return [dd,mm,yr].join("/") + " " + [hr,mn,ss].join(":");
35.     }
36.
37.     function prepareDOM()
38.     {
39.         var rooms = getRooms();
40.         showRooms(rooms);
41.     }
42.
43. </script>

```

One of the contained functions getRooms() adds a Jaxer-specific property on the function, getRooms.proxy = true; so it can be called from the browser. We could have used an alternate syntax: getRooms.runat = "server-proxy"; .

The `prepareDom()` function is invoked later on the page by being bound to the special Jaxer event `onserverload`. This event is fired when the server has completed loading of the page and has a DOM available for manipulation by JavaScript code.

The `onserverload` event is the server-side equivalent of the traditional client side `onload`.

```
1. <body onserverload="initAuthentication(); prepareDOM()">
```

At this point the authentication is initialized, and the `prepareDom()` function is called. This will insert rows displaying any existing chat rooms into the DOM contents of the home page before sending the page to the client. It does this by querying the database table `messages` (lines 7-9) and returning a distinct set of rooms as well as information on when each room was opened and when the last message was posted. This content also contains a hyperlink to enter the chatroom (shown below).

Recent Chat Rooms:

Room 1 Started: 2007-09-04 18:53:56 Last message: 2007-09-04 18:54:05

Room 3 Started: 2007-09-04 18:55:19 Last message: 2007-09-04 18:55:19

☐ Open rooms in new window/tab

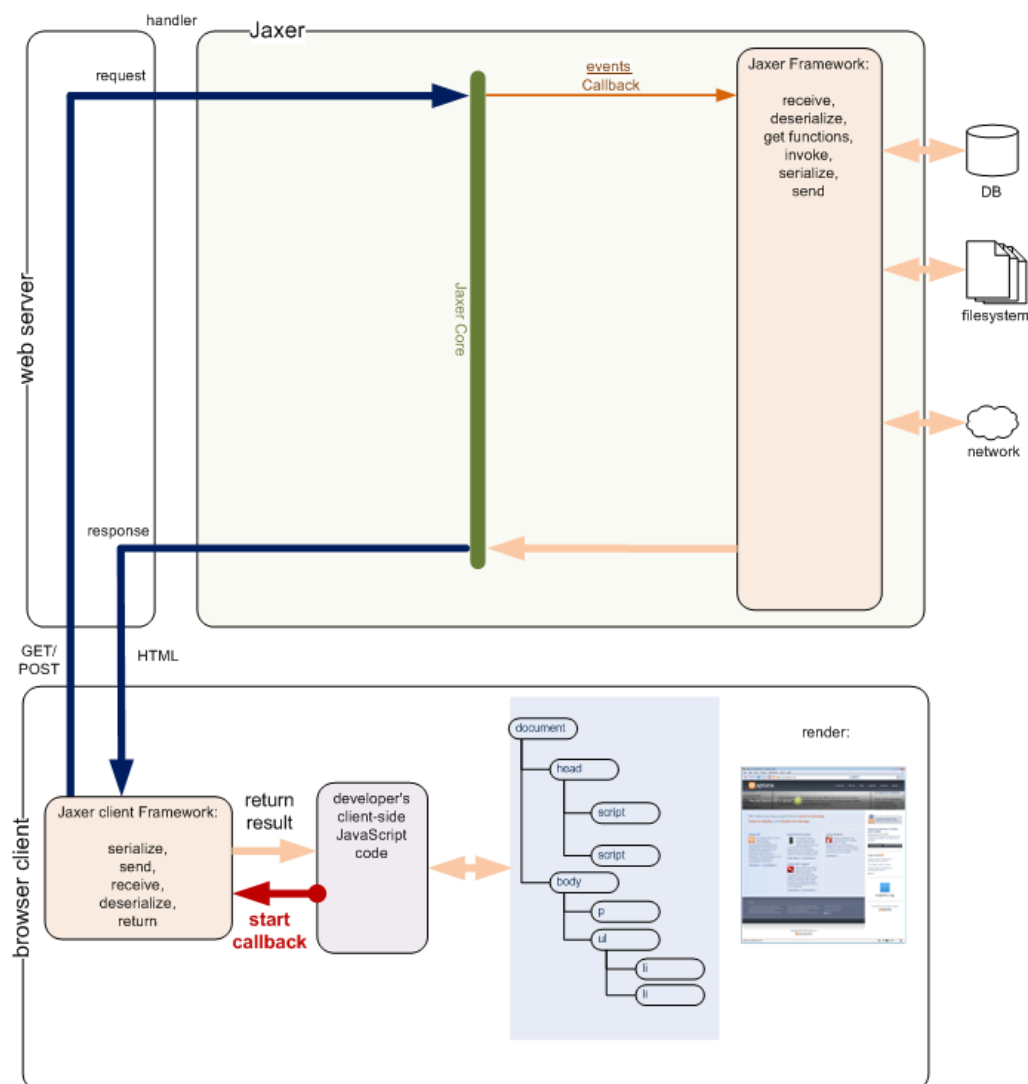
Now let's turn to the callback mechanism used by the Jaxer chat app.

Callbacks made easy

When you enter the application for the first time, no room will be listed and you will have to create a new room.

However, before you can create any new rooms, you first have to login to chat. If this is your first chat visit, you will need to create a new account before you can login.

The next two sections discuss what happens on both the client and the server when you login to chat (assuming that you already have an account created). Here is a detailed diagram of the process flow:



On the client

The chat login process makes extensive use of callbacks from the client to the server to provide the services required to authenticate users. The following code demonstrates some of the unique capabilities of the Jaxer server with respect to remote function invocation or callbacks.

The `login` function (located in `_login.js`) invokes a local (client-side) function `checkCredentials(username, password)` which is the proxy for the previously-shown server-side `checkCredentials` function.

```

01. function login()
02. {
03.     var username = $('#username').value;
04.     var password = $('#password').value;
05.     var username = checkCredentials(username, password);
06.     if (username != "")
07.     {
08.         fromTemplate('loginComponent', 'loginAuthenticated');
09.         setTimeout($('#authenticatedUsername').innerHTML = '' + username + '', 0);
10.         changeAuthentication(true);
11.     }
12.     else
13.     {
14.         $('#loginMessage').innerHTML = "Sorry, try again";
15.     }
16. }
17. login.runat = "client";

```

On the server

The `checkCredentials` function can only truly execute on the server, but since it's marked to be proxied, the client will also get a function called `checkCredentials` (shown above) which simply provides a remote wrapper for the server code. Internally it does this by serializing its arguments to a JSON format and passing them to the server, where they are deserialized and the server-side function is invoked. When the server-side function completes, its result is serialized to JSON and then passed back to the client, where it's deserialized and returned as the result of the proxy as if the proxy function had carried out the processing.

```

01. function checkCredentials(username, password)
02. {
03.     var rs = Jaxer.DB.execute("SELECT * FROM users WHERE username = ? AND password = ?",
04.         [username, password]);
05.     if (rs.rows.length == 0)
06.     {
07.         return "";
08.     }
09.     var user = rs.rows[0];
10.     makeAuthenticated(user);
11.     return user.username;
12. }
13. checkCredentials.proxy = true;

```

One point worth noting is that this is not asynchronous, and the client will wait for the response. For every server-side function to be proxied Jaxer adds a method to the function for asynchronous remote invocation. , for example `checkCredentials` and `checkCredentials.async()` .

This callback architecture is the key to providing the seamless client server communications without any of the complexity of the more traditional XHR techniques normally associated with Ajax functionality. With Jaxer it is all just Javascript and the boundary between client and server can be smooth and simple.

Inside the Jaxer chat application

You should now be familiar with the functionality of the Jaxer chat application. The next section discusses some of the key technical differences between how a typical web application is constructed and the way the demo chat application uses the unique capabilities of the Jaxer Server.

Architectural distinction

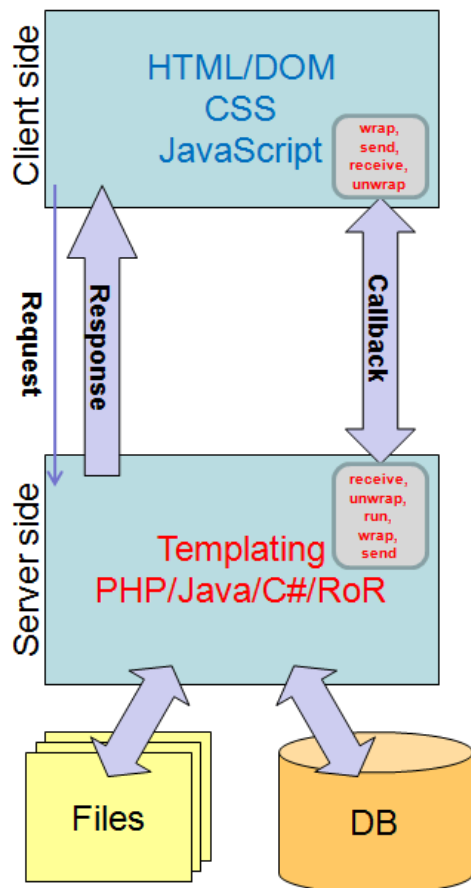
The main architectural difference of the Jaxer server from other page composition engines, such as PHP, JSP and ASP, is that both the client side and server side use essentially the same platform. This means that you can use familiar DOM manipulation techniques with standard JavaScript tools and libraries on both the client and server side. As a developer, you benefit from this approach by no longer having to deal with complex paradigms, such as emitting JavaScript from Java. Using Jaxer, you can even invoke server-side functionality directly from the client in a completely seamless manner.

The Jaxer server wraps typical server-side functionality within a tightly controlled namespace (the `Jaxer` object) to prevent variable and scope conflict with other Javascript libraries and tools.

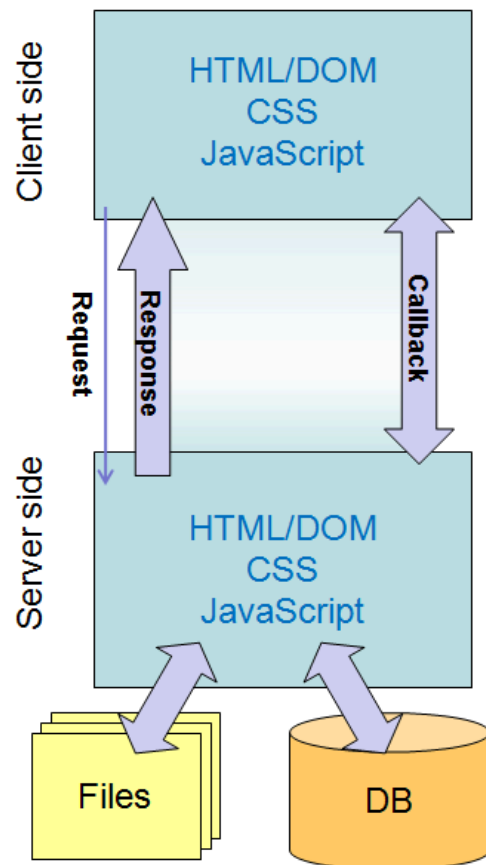
Side-by-side comparison

The following diagram illustrates how Jaxer can replace the common server technologies of most dynamic web applications.

Traditional Application Stack



Jaxer Application Stack



With the Jaxer server, you can perform all the usual activities associated with web applications, such as database access, session management, error logging and file system access.

In the next few sections of this document, you will learn about some of the unique features of the Jaxer Server and examine at how they were implemented for the Jaxer chat application.

Jaxer runat and proxy reference

One of the initial differences you will notice between the Jaxer source and typical web applications is the addition of the `runat` attributes to the script tags in the HTML markup. These attributes tell the Jaxer server how to manage the script code. Because the server can run JavaScript on the server-side, client-side, or both, and can inject proxies into the client, this meta information instructs the server how to handle the scripts appropriately.

Basic runat values

The following table describes the standard `runat` attributes that are applied to the `<script>` tag.

value	description
client	The functions and code contained in the script block will run in the client browser only. This functions exactly as a regular script block. This is the default value of the <code>runat</code> attribute, so usually you'll omit it for <code>script</code> blocks intended for the client. Its main use is to override the <code>runat</code> attribute of a specific function within a server-side script block. <i>Note: if a script block has <code>runat = "client"</code> (or no <code>runat</code> attribute), it will not run at all server-side, so you cannot override the <code>runat</code> behaviors of individual functions from within this block.</i>
server	The functions and code contained in the script will run on the server only. Any functions defined within the script block will be cached in association with this page. These functions are not directly callable from the client, but they can be called during callback processing by other server-side functions. These script blocks will not be presented to the client browser.
both	The functions and code contained in the script will run on both the client and the server. Any functions defined within the script block will be cached in association with this page. The server-side functions are not directly callable from the client, but they can be called during callback processing by other server-side functions.

Example The following code snippet is an excerpt from the `index.html` page in the Jaxer chat application folder. This example shows an external script being included on the page for server-side-only execution.

```
1. <script type="text/javascript" src="http://jaxer.org/tutorials/jaxer-include/_login.js" runat="server"></script>
```

Advanced runat values

The following `runat` values can also be used on the script tags. We expect most use cases would be covered by the basic attributes shown above.

value	description
server-proxy	Same as the basic 'server' target except ALL the functions will be proxied by default
server-nocache	Same as the basic 'server' target except NONE of the functions will be cached by default
both-proxy	Same as the basic 'both' target except ALL the functions will be proxied by default

both-nocache Same as the basic 'both' target except NONE of the functions will be cached by default

Programmatic runat configuration

Jaxer is aware of some special function object properties that can be declared on individual function objects to control how they are managed. When these are specified, the property value will override the containing script block's runat setting for the individual function. This allows more granular control and prevents the need to break scripts out into separate blocks or files depending on their desired runat value.

property

description

proxy

Server-side functions can be declared to be proxied so they are callable from the client side. This is achieved by specifying a proxy property on the function object. The possible values for this property are true or false . This is only required for enabling the proxying of the function. By default, in a `<script runat="server">` block, the functions are not proxied. Note that if a function is not proxied, it isn't just that proxies are not inserted into the client to facilitate calling it: it's actually marked as not callable on the server, so hacking the client to try to call the function on the server will not work.

runat

Takes the same values as the `<script>` tag runat attributes.

Example

The `_login.js` file referenced in the example above contains some functions that explicitly override the runat='server' directive specified on the script tag used to load the file.

In this snippet the function will be proxied:

```
1. function removeCredentials()
2. {
3.     // Jaxer 0.9.7 or earlier // Jaxer.session.remove("username");
4.     delete Jaxer.session.username;
5. }
6. removeCredentials.proxy = true;
```

In this snippet the function will run client side:

```
01. function validate(validationFunction, candidate)
02. {
03.     if (candidate == "") return; // Not done filling this out, perhaps
04.     var error = validationFunction(candidate);
05.     if (error != "")
06.     {
07.         alert(error);
08.     }
09. }
10. validate.runat = "client";
```

Recommended style

The following illustrates one simple way of using the runat and proxy options in a typical code scenario. We choose to group all the server-side code in one script block, and explicitly designate a subset of function to be proxied. Then all client-side code goes in a different script block (where there isn't even the option of programmatically changing it by setting a different runat or proxy value). Of course you may choose a different way of organizing your code if that makes more sense. And for large amounts of code, it may also make sense to extract the code into (reusable) external JavaScript files.

```
01. <script type="text/javascript" runat="server">
02.     function setPassword(username, newPassword)
03.     {
04.         // put code in here to directly set the password of a given username
05.         // this code should not be callable from the client
06.     }
07.
08.     function changePassword(username, oldPassword, newPassword){
09.         // put code in here to first verify the submitted password,
10.         // and then -- if successful -- call setPassword to actually make changes
11.         // this code should be callable from the client
12.     }
13.     changePassword.proxy = true;
14. </script>
15.
16. <script type="text/javascript">
17.     function submitPassword()
18.     {
19.         // put client-side code here to grab the username and old and new passwords
20.         // and call changePassword on the server
21.     }
22. </script>
```

Alternate Syntax

Jaxer provides a useful convenience array inside the Jaxer namespace to allow the proxy functions to be declared in a single group within your Javascript code:

```
Jaxer.proxies = [myFunc1, myFunc2, "myFunction"];
// ...
Jaxer.proxies.push(myFunc3, "myFunction4");
```

The value of Jaxer.proxies at the end of server-side page processing (i.e. when proxies are about to be inserted) determines which server-side functions will have their proxy property be set to true . You can also use this to remove all proxied functions by setting the value to null .

Note: Jaxer.proxies is NOT a complete collection of the functions being proxied by the server, it is just a convenient way to express the myFunc.proxy = true; syntax for multiple function references.

<jaxer:include> reference

Jaxer provides several special <jaxer:include> tags for implementing server-side include statements.

Why use server-side includes?

If you are managing a web site that has more than a few pages, you may find that making changes to all those pages can be tedious, particularly if you are trying to maintain a standard functionality or look across all of your pages.

Using an include file for a header and/or a footer can reduce the burden of these updates. You just have to make one footer file, and then include it into each page with the include SSI command. The `<jaxer:include>` element can determine what file to include by the specified `src` attribute. The value of the `src` attribute should be a URL relative to the document being served, or an absolute URL starting with a `/` representing the web server's document root. Alternatively, you can use a path attribute instead of the `src` attribute; its value is an absolute path in the filesystem. In any case, the included file must be on the same server as the page being served.

Jaxer also provides `Jaxer.load(src)` to programatically load and evaluate (server-side) a JavaScript file.

Examples

The following examples show how to include a file from the server's local file system as part of the current document.

From within the HTML of the document.

```
1. <jaxer:include src="_login.html"></aptana:include>
```



From within a Javascript file.

```
1. Jaxer.load("setup.js");
```



This functionality is similar to the php `include` directive or the Apache `#include` functionality.